Product Requirements Document

CARLA Autonomy
Demo & Research Platform

A Simulation-Based Autonomous Driving Visualization System

Version	v1.0 — Initial Release
Status	DRAFT — Engineering Review
Classification	Confidential — Internal Only
Scope	Simulation / Research Demo — Not a production AV
Simulator	CARLA 0.9.15 (Unreal Engine 4.26)
Primary Language	Python 3.10 + PyTorch 2.x

> ⚠ **Scope Boundary:** This PRD defines a research and visualization system running entirely inside the CARLA simulator. It does not specify requirements for production AV deployment, real-world vehicle integration, or regulatory compliance.

1 Product Overview

1.1 Purpose

This platform is an autonomous driving research and visualization tool built on top of the CARLA simulator. Its purpose is to demonstrate, study, and communicate how a modern AI autonomy stack works — from raw sensor data through perception, prediction, and planning, to vehicle control — in a visually compelling, reproducible, and modular way.

The platform is inspired by the public-facing technology demonstrations published by Waymo, Nuro, Cruise, and Tesla: real-time bird's-eye views of the world model, translucent bounding boxes around detected agents, colored trajectory overlays, and live sensor feeds. The goal is to build that kind of demo experience on top of a fully instrumented, programmable simulation environment where every component of the stack can be inspected, modified, and evaluated.

This is NOT a production self-driving car system. There is no vehicle hardware, no regulatory compliance layer, and no safety-of-life requirement. It IS a technically faithful research platform that implements the same architectural patterns used in real autonomy stacks, but within the safe, controlled environment of a simulator.

1.2 What the Platform Does

- Simulates a fully instrumented ego vehicle navigating through configurable CARLA environments.
- Streams virtual sensor data (camera, LiDAR, radar, GNSS/IMU) from the simulated vehicle.
- Runs a modular autonomy stack: perception → environment model → prediction → planning → control.
- Renders a real-time visualization dashboard showing perception outputs, tracked objects, predicted trajectories, and the planned ego path — styled like commercial autonomy demos.
- Supports multiple driving scenarios: urban streets, highways, intersections, construction zones, pedestrian crossings, dynamic traffic, and emergency vehicle interactions.
- Provides a scenario scripting system so new scenarios can be defined in JSON/Python without modifying the core stack.
- Records all sensor data, ground truth, and stack outputs for offline replay and evaluation.

1.3 Target Users

User	Responsibilities	Relevant Sections
Perception Engineers	Sensor pipeline, detection models, tracking	3 (Perception), 5, 7, 8
Planning Engineers	Behavior FSM, trajectory generation, cost functions	3 (Planning), 4, 5, 7
Visualization Engineers	Dashboard UI, rendering pipeline, data streaming	2, 6, 7, 8
ML / Research Engineers	Model training, dataset generation, evaluation	5, 8, 10

Demo / Product Stakeholders	Platform capabilities, scenario coverage, milestones	2, 4, 9, 10

1.4 Success Criteria

1. Ego vehicle successfully completes all 7 defined scenario types without collision in ≥ 85% of runs.
2. Perception achieves mAP ≥ 0.75 on CARLA ground-truth labels for vehicles, cyclists, and pedestrians.
3. Visualization dashboard renders at ≥ 20 FPS on RTX 3080 or equivalent with all overlays active.
4. Any scenario can be launched via a single CLI command with a JSON config file.
5. Full pipeline — from sensor data to vehicle control command — runs at ≤ 100ms end-to-end latency.
6. All scenario runs are fully replayable using recorded data without re-running CARLA.

2 Key User Experience

2.1 Visualization Philosophy

The visualization must communicate three distinct categories of information simultaneously and without ambiguity:

Category	What It Shows	Visual Treatment
Perception	What the AI currently detects in the world	Sharp outlines, bounding boxes, solid labels
Prediction	What the AI expects other agents to do next	Semi-transparent arcs, faded trail lines
Planning	What the AI intends to do	Solid highlighted spline on the road surface

All three layers are rendered simultaneously in the primary view. Color and opacity encode information type, not just aesthetic choice. Engineers and stakeholders must be able to read the visualization and immediately understand which layer they are looking at.

2.2 Dashboard Layout

The dashboard consists of three panels:

Panel	Content
LEFT — World View (55% width)	Top-down 3D bird's-eye rendering of the CARLA environment. Ego vehicle centered and highlighted. Surrounding agents rendered as simplified translucent 3D blocks. Detected objects annotated with bounding boxes. Temporary and permanent lane boundaries drawn as colored lines. Drivable space shown as a pale overlay. Planned ego path rendered as a solid cyan spline. Predicted NPC trajectories shown as faded arcs.
TOP RIGHT — Ego Camera Feed (45% × 60% height)	Forward-facing RGB camera view from the ego vehicle. Overlaid with detection bounding boxes, lane boundary lines, traffic signal annotations, and a confidence score chip per detection. HUD bar at bottom shows: speed (mph), behavior state, perception latency, and active scenario name.
BOTTOM RIGHT — LiDAR / Sensor Panel (45% × 40% height)	Real-time LiDAR point cloud rendered as colored dots (intensity or height-mapped). Optionally switchable to: radar return view, semantic segmentation output, or a data telemetry graph showing stack latency per module over time.

2.3 World View Rendering Specification

Element	Visual Specification

Ego vehicle	Solid white body, 2px bright cyan border (#00E5FF), always centered
Other vehicles	Translucent grey fill rgba(180,180,200,0.4), 1px white outline, class label chip
Cyclists	Translucent amber fill rgba(251,191,36,0.3), 1px amber outline
Pedestrians	Translucent green fill rgba(34,197,94,0.3), 1px green outline
Traffic cones	Orange marker (#FF6D00), 30% opacity influence radius circle
Permanent lane markings	Solid white, 1px, 0.5 opacity
Temporary lane boundaries	Dashed white, 2px, 0.8 opacity
Drivable space overlay	Pale blue fill rgba(30,136,229,0.12)
Ego planned path	Solid cyan spline (#00BCD4), 3px, rendered on road surface
Predicted NPC trajectories	Dashed colored arcs, 1px, 0.45 opacity, color matches agent class
Traffic signals	Icon badge: red / amber / green disc above stop line
Detection bounding boxes	1.5px colored outlines, class + confidence chip in dark background

2.4 HUD Elements

HUD Field	Data Source and Format
Speed	CARLA telemetry · display in mph, 1 decimal
Behavior state	Planner FSM output · e.g. LANE_KEEP / MERGING / STOPPING_FOR_RED
Active scenario	Scenario config name · e.g. 'CZ-01: Construction Zone'
Perception latency	Wall-clock inference time · display in ms
End-to-end latency	Time from sensor tick to control command · display in ms
Detected object count	Live count per class: vehicles / cyclists / pedestrians / cones
Sensor status	Camera OK / LiDAR OK / Radar OK / GNSS OK badge row

Planning confidence	Planner trajectory cost score 0.0–1.0

3 System Architecture

3.1 Architecture Principles

Modularity: Each layer exposes a typed Python interface. No layer may import directly from a non-adjacent layer. All inter-layer data is passed as typed dataclasses or NumPy arrays.

Testability: Every layer must be independently testable using synthetic fixture inputs without CARLA running. This is enforced via a fixtures/ directory of pre-recorded sensor frames.

Replay-First: All sensor data and stack outputs are logged per-frame to a compressed replay archive. Any run can be replayed, paused, scrubbed, and inspected without re-running CARLA.

3.2 Layer Specifications

3.2.1 Simulation Layer

Responsibility: Provide a physics-accurate, sensor-accurate 3D world that serves as the authoritative ground truth environment.

Attribute	Specification
Simulator	CARLA 0.9.15 (UE4.26 backend)
Primary maps	Town04 (highway / merge zones), Town05 (urban grid), Town10HD (dense urban, intersections)
Tick rate	Fixed 50ms physics tick (20 Hz)
NPC traffic	CARLA TrafficManager — configurable vehicle count, speed variance, lane-change aggression
Pedestrian simulation	CARLA walker AI — configurable density, crossing behavior, speed
Weather presets	ClearNoon (primary), CloudySunset (secondary), WetCloudy (tertiary)
Scenario scripting	Python carla.Client API + JSON scenario config. Spawns actors, sets routes, triggers events.
Ground truth export	Per-frame JSON: all actor poses, velocities, 3D bounding boxes — used for perception evaluation

3.2.2 Sensor Layer

Responsibility: Attach virtual sensors to the ego vehicle and stream raw data to the perception layer at the configured rate.

Sensor	Configuration	Notes

RGB Camera (front)	1280×720, 90° FOV, 20 FPS, hood-mount +2.3m fwd +0.8m up	Primary perception input
RGB Camera (rear)	640×480, 120° FOV, 10 FPS	Used for blind-spot awareness
RGB Camera (left/right)	640×480, 90° FOV, 10 FPS	Side-mirror equivalent
LiDAR	64 channels, 80m range, 1M pts/sec, 10 Hz rotation, roof center	Primary 3D input
Radar	50m range, 30° azimuth, 20 Hz, front bumper	Velocity estimation, fog robustness
Semantic Seg Camera	Same config as RGB front — pseudo-labels only	Offline eval use only
GNSS	Clean CARLA ground truth position at 10 Hz	Localization input
IMU	Accelerometer + gyroscope at 100 Hz	LiDAR motion compensation

3.2.3 Perception Layer

Responsibility: Detect, classify, and track all relevant objects and road features from raw sensor streams. Output typed detection lists with 3D positions, classes, confidences, and track IDs.

Object Detection (Vehicles, Cyclists, Pedestrians)

- 2D detection: YOLOv8-medium on RGB front camera (primary) and side cameras (secondary).
- 3D bounding boxes: PointPillars (PyTorch) on LiDAR point cloud. Fused with 2D detections via projection-based association.
- Output: List[ObjectDetection] — each with: (track_id, class, world_bbox_3d, velocity_estimate, confidence).
- Latency target: ≤ 60ms per frame on RTX 3080.

Lane and Road Boundary Detection

- Model: Ultra-Fast Lane Detection v2 on front camera.
- Output: List[LaneLine] — each with: (lane_id, polyline_image, polyline_world, type: SOLID|DASHED|TEMPORARY, confidence).
- Road boundaries (curbs, guardrails): derived from LiDAR ground-plane segmentation using RANSAC + height threshold.

Drivable Space Estimation

- SegFormer-B0 semantic segmentation on front camera. Labels: road, lane_marking, cone, vehicle, pedestrian, sidewalk, other.
- Output: DrivableSpaceMask (H×W binary) + per-pixel class probabilities.

Traffic Light Detection

- YOLOv8-small fine-tuned on CARLA traffic light images. Detects light position + state (RED/AMBER/GREEN/UNKNOWN).

- Output: List[TrafficLightDetection] — each with: (world_xyz, state, stop_line_distance, confidence).

Cone Detection

- YOLOv8-small for 2D detection, 3D position via LiDAR-depth association (DBSCAN cluster centroid within projected bbox region).
- Output: List[ConeDetection] — each with: (world_xyz, confidence).

Multi-Object Tracking

- SORT tracker applied to all detected objects across frames. Provides stable track IDs and velocity estimates.
- Track lifecycle: TENTATIVE (1–3 frames) → CONFIRMED (≥4 frames) → LOST (missed for ≥5 frames) → DELETED.
- Only CONFIRMED tracks are passed to the prediction and planning layers.

3.2.4 Localization Layer

Responsibility: Maintain an accurate estimate of the ego vehicle's pose (position + orientation) relative to the map.

- In simulation: GNSS ground truth provides perfect position. This layer exists primarily to define the interface for future real-world integration.
- Map alignment: Ego pose is matched to the nearest OpenDRIVE road segment using projection onto the lane graph. Outputs: current lane ID, lane-relative position (s, d in Frenet coordinates), heading error.
- Output: EgoPose dataclass — (world_xyz, yaw, speed, acceleration, current_lane_id, frenet_s, frenet_d).

3.2.5 Environment Model (Mapping) Layer

Responsibility: Combine the static HD map (OpenDRIVE) with real-time sensor observations to produce a dynamic local map that represents the world as the AI currently understands it.

Component	Description
Static map	OpenDRIVE XML from CARLA → lane graph (nodes = lane segments, edges = connections, attributes = speed limit, turn type, traffic control)
Detected agent integration	All CONFIRMED tracked objects inserted into the environment model with their current pose and velocity
Cone accumulation	5-second rolling window of cone detections, RANSAC-filtered, clustered into unique cone instances
Temporary lane inference	≥3 cones in a lane region → fit polyline through cone positions → temporary boundary overlaid on lane graph
Lane closure detection	Cone density ≥ 1 per 4m for ≥ 20m along a lane → mark lane as CLOSED in the graph
Traffic light state integration	Traffic light detections merged into map node attributes for stop-line decision making

Output	LocalMap dataclass: { static_lanes, dynamic_agents, cone_instances, temporary_boundaries, closed_lanes, traffic_signal_states }

3.2.6 Prediction Layer

Responsibility: Generate short-horizon motion forecasts for all tracked agents. These forecasts are consumed by the planner as constraints.

Attribute	Specification
Agents predicted	All CONFIRMED tracked vehicles, cyclists, and pedestrians
Model (baseline)	Constant-velocity kinematic model — fast, interpretable, sufficient for highway and sparse scenarios
Model (advanced)	Social Force Model for dense pedestrian interactions. Optional: LSTMPredictor trained on CARLA-recorded trajectories.
Prediction horizon	5 seconds at 10 Hz = 50 steps per agent
Uncertainty	Gaussian uncertainty ellipses grow with horizon. Used by planner as soft constraints.
Output	List[AgentPrediction]: each with (track_id, class, predicted_trajectory: List[(x,y,t)], confidence_by_step)

3.2.7 Planning Layer

Responsibility: Generate a collision-free, comfortable trajectory for the ego vehicle that obeys traffic rules, respects agent predictions, and achieves the current driving goal.

Behavior Planner

Implemented as a hierarchical finite state machine (FSM). States and transitions:

State	Entry Condition	Exit Condition
LANE_KEEP	Default state	Lane closed ahead / intersection / target reached
PREPARE_MERGE	Closed lane detected 60m ahead	Gap in target lane confirmed by prediction
MERGING	Gap confirmed, begin lateral move	Ego fully within target lane bounds
INTERSECTION_APPROACH	Intersection node within 40m	Stop line reached or intersection cleared

STOPPING_FOR_RED	Traffic light = RED within 50m	Light turns GREEN
PEDESTRIAN_YIELD	Pedestrian predicted to enter path	Pedestrian path cleared
EMERGENCY_YIELD	Emergency vehicle detected (siren / class)	Emergency vehicle passed
CONSTRUCTION_NAVIGATE	Construction zone markers detected	Zone exit marker detected
GOAL_REACHED	Within 5m of goal waypoint	Scenario ends

Motion Planner

- Algorithm: Frenet-frame polynomial trajectory generation (quintic lateral + quartic longitudinal) over a 5-second horizon.
- Candidate generation: Sample N=50 candidate trajectories varying lateral offset (−3m to +3m in 0.5m steps) and target speed (0 to speed_limit in 2.5 mph steps).
- Cost function: cost = (collision_risk × 100) + (cone_proximity × 40) + (deviation_from_center × 5) + (jerk × 2) + (speed_error × 3) + (traffic_violation × 1000).
- Hard constraints: zero penetration of any detected agent bounding box (inflated by 0.5m safety margin). Ego must stay within lane boundaries (or temporary boundaries if active).
- Output: EgoTrajectory — List[(x, y, yaw, velocity, timestamp)] at 10 Hz for 5 seconds.

3.2.8 Control Layer

Responsibility: Convert the planned trajectory into low-level CARLA vehicle commands at 20 Hz.

- Lateral: Stanley controller tracking trajectory waypoints. Gain Kp=0.5 (tunable via config).
- Longitudinal: PID speed controller. Kp=0.3, Ki=0.05, Kd=0.02 (tunable). Anti-windup: integrator clamped to ±1.0.
- Emergency brake: If predicted collision TTC < 1.5 seconds, override trajectory with full brake command.
- Output: carla.VehicleControl(throttle, steer, brake) applied at each 50ms physics tick.

3.2.9 Visualization Layer

Responsibility: Render all perception, prediction, and planning outputs into the dashboard. See Section 6 for complete UI specification.

- Data source: Visualization layer subscribes to typed outputs from all upstream layers via a pub/sub event bus (ZeroMQ or in-process Python queues).
- Rendering: Pygame (primary — bird's-eye view + HUD) + OpenCV (camera + LiDAR panels). Optional Three.js dashboard for web-based viewing.
- Render rate: 20 FPS target, decoupled from the 20 Hz autonomy pipeline via a dedicated render thread.

4 Scenario System

4.1 Scenario Architecture

A scenario is a self-contained JSON configuration file that specifies: the CARLA map, ego spawn point and goal, NPC actor spawns and behaviors, static prop placements, event triggers, and evaluation criteria. The scenario runner loads this config and orchestrates CARLA accordingly. No Python code changes are required to add a new scenario.

```
python run_scenario.py --config scenarios/CZ-01_construction_zone.json --visualize --record
```

4.2 Scenario Configuration Schema

```
{ "scenario_id": "CZ-01", "map": "Town04", "ego_spawn": { "x": 100, "y": 50, "yaw": 0 },
  "ego_goal": { "x": 300, "y": 50 }, "max_duration_s": 120,
  "npcs": [{ "model": "vehicle.tesla.model3", "spawn": {...}, "route": [...] }],
  "props": [{ "type": "trafficcone01", "x": 180, "y": 48 }],
  "triggers": [{ "type": "lane_closure", "at_s": 80, "lane_id": 1 }],
  "eval": { "min_completion_rate": 0.9, "max_collisions": 0 } }
```

4.3 Defined Scenarios

ID	Scenario Name	Key Challenge	Primary Evaluation Metric
SC-01	Highway Cruise	Lane keeping, speed control, safe following distance at 65 mph	Collision-free completion, speed adherence ±5 mph
SC-02	Urban Intersection (4-way)	Stop sign compliance, yield to cross-traffic, pedestrian detection at crosswalk	Stop-line stop, no pedestrian incursion
SC-03	Lane Merge (Highway On-Ramp)	Gap detection in fast traffic, smooth merge, no abrupt braking	Merge completion, no emergency brakes in following vehicles
SC-04	Pedestrian Crossing	Yield to pedestrians mid-block and at crosswalks, re-accelerate cleanly	No pedestrian proximity violation < 2m
SC-05	Traffic Light Grid	Red light stops, green light proceed, amber light decision	Zero red-light violations, no unnecessary stops on green
SC-06	Construction Zone (CZ-01)	Cone detection, temporary lane inference, merge past closed lane	Cone-avoidance, smooth merge, completion rate ≥ 90%

SC-07	Emergency Vehicle Response	Detect emergency vehicle (siren class label), pull to right, yield path	Pull-over within 5 seconds of siren detection

4.4 Adding New Scenarios

7. Create a new JSON file in scenarios/ following the schema in 4.2.
8. If a new NPC behavior is needed, implement it as a Python class in scenario_agents/ and reference it in the JSON.
9. If a new event trigger is needed, add a handler to scenario_runner.py (< 50 lines typically).
10. Run the scenario with --validate to check it runs to completion without errors.
11. Add evaluation criteria to the JSON eval block. The eval harness will automatically measure them.

5 Core Features

5.1 Feature Inventory

Feature	Description	Implementing Layer
Real-time perception visualization	All detections rendered as overlays on camera feed and bird's-eye view at ≥20 FPS	Visualization Layer
Multi-class object detection	Vehicles, cyclists, pedestrians, cones, traffic lights detected and classified	Perception Layer
Multi-object tracking	Stable track IDs across frames with velocity estimation via SORT	Perception Layer
Lane detection	Permanent and temporary lane boundaries extracted from camera and map	Perception Layer
Drivable space estimation	Semantic segmentation of road surface, used by planner as soft constraint	Perception Layer
Traffic light detection	State detection (RED/AMBER/GREEN) with stop-line distance	Perception Layer
Cone-based temporary lane inference	≥3 cone detections trigger boundary override in lane graph	Mapping Layer
Behavior state machine	FSM with 9 states covering all defined scenarios	Planning Layer
Frenet trajectory planning	Polynomial candidate generation with cost-function selection	Planning Layer
Agent trajectory prediction	5-second multi-agent forecasts with uncertainty	Prediction Layer
Sensor fusion	LiDAR-camera association for 3D position of 2D detections	Perception Layer
Multi-scenario system	JSON-driven scenario config, CLI launch, 7 predefined scenarios	Simulation Layer
Replay and recording	Per-frame sensor + stack output logging, offline replay without CARLA	All Layers
Configurable sensor setup	Sensor types, positions, and resolutions set in YAML config	Sensor Layer

Evaluation harness	Per-scenario metrics computed automatically at end of run	Scenario System
Optional web dashboard	Three.js / React dashboard consuming WebSocket data stream	Visualization Layer

6 Visualization Design

6.1 Rendering Architecture

The visualization system runs as a dedicated process or thread that consumes typed output events published by the autonomy stack. It is entirely read-only relative to the autonomy pipeline — it cannot modify planning or control commands.

Component	Technology
Bird's-eye world view	Pygame surface — ego-centered, top-down orthographic projection
Camera feed panel	OpenCV imshow or Pygame surface — raw RGB + cv2 overlay rendering
LiDAR panel	Open3D Visualizer or Pygame surface — colored point cloud (height or intensity mapped)
HUD overlay	Pygame text rendering — fixed-position HUD elements over camera panel
Optional web dashboard	React + Three.js — WebGL 3D scene consuming WebSocket JSON stream from a Flask/FastAPI server

6.2 Bird's-Eye View Render Pipeline

12. Draw road polygons from static map (dark grey fill, white lane line edges).
13. Draw drivable space overlay (pale blue, 12% opacity) from SegFormer output projected to ground plane.
14. Draw permanent lane markings (white lines, 50% opacity).
15. Draw temporary lane boundaries from cone inference (dashed white, 80% opacity).
16. Draw predicted NPC trajectories (faded dashed arcs, color by agent class, 45% opacity).
17. Draw ego planned path (solid cyan spline, 3px, 100% opacity, on road surface).
18. Draw NPC agent boxes (translucent fill per class color, white outline, class+ID chip).
19. Draw traffic light badges (disc icon above stop line node).
20. Draw ego vehicle (white body, cyan border, directional arrow).
21. Draw HUD overlay (speed, behavior state, latency, scenario name).

6.3 Camera Panel Overlay Pipeline

22. Display raw RGB frame from front camera.
23. Draw detection bounding boxes: 1.5px colored outline per class, confidence chip in dark rounded rectangle.
24. Draw lane boundary lines projected from 3D world to image plane.
25. Draw traffic signal annotation: state disc above the detected signal crop.
26. Draw drivable space mask as a semi-transparent colored overlay.
27. Draw HUD bar at bottom: speed, behavior state, latency ms, sensor status badges.

6.4 LiDAR / Sensor Panel

Default view: LiDAR point cloud rendered as height-mapped colored dots (blue = ground, red = high). Each point colored by z-height relative to ground plane.

Switchable views (keyboard shortcut):

- Height map (default)
- Intensity map
- Semantic segmentation output (camera)
- Radar return overlay on camera
- Stack latency telemetry graph (rolling 10-second window, per-module breakdown)

6.5 Interaction Controls

Key / Control	Action
SPACE	Pause / resume simulation
R	Start / stop recording
1–7	Switch active scenario
TAB	Cycle sensor panel view (LiDAR / seg / radar / telemetry)
F	Toggle full-screen world view
P	Toggle prediction arc visibility
L	Toggle lane detection overlay
ESC	Quit
Mouse scroll	Zoom bird's-eye view
Mouse drag	Pan bird's-eye view (when not ego-centered)

7 Data Flow

7.1 Pipeline Overview

The full data flow from sensor to vehicle command follows a strict left-to-right dependency chain. Each stage outputs a typed data structure consumed by the next. Stages are executed sequentially within a single pipeline tick triggered by the CARLA sensor callback.

Stage	Input	Output	Latency Budget
Sensor Layer	CARLA callbacks (camera, LiDAR, radar, GNSS, IMU)	Raw frames, point clouds, IMU readings — buffered to shared memory	0ms (async)
Perception Layer	Latest camera frame + LiDAR scan + IMU	List[ObjectDetection], List[LaneLine], DrivableSpaceMask, List[TrafficLightDetection], List[ConeDetection]	≤60ms
Localization Layer	GNSS + IMU + map	EgoPose (xyz, yaw, speed, lane_id, frenet_s, frenet_d)	≤5ms
Environment Model	Perception outputs + EgoPose + static map	LocalMap (agents, lanes, cones, temp_boundaries, closed_lanes, signals)	≤10ms
Prediction Layer	LocalMap (agent list + velocities)	List[AgentPrediction] (5s trajectory per agent)	≤15ms
Planning Layer	LocalMap + EgoPose + AgentPredictions	EgoTrajectory (5s, 10Hz waypoints)	≤20ms
Control Layer	EgoTrajectory + EgoPose	carla.VehicleControl (throttle, steer, brake)	≤5ms
Visualization Layer	All above outputs (pub/sub)	Rendered frames on screen	Async, ≥20FPS

7.2 Data Structures

ObjectDetection

```
@dataclass
class ObjectDetection:
    track_id: int
    object_class: str          # 'vehicle' | 'cyclist' | 'pedestrian' | 'cone' |
'traffic_light'
    world_bbox_3d: np.ndarray  # shape (8,3) — 8 corners in world coordinates
    velocity: np.ndarray       # (vx, vy, vz) m/s
    confidence: float          # 0.0–1.0
    track_state: str           # 'TENTATIVE' | 'CONFIRMED' | 'LOST'
```

EgoTrajectory

```
@dataclass
class EgoTrajectory:
    waypoints: List[Waypoint]  # 50 points at 0.1s intervals
    cost: float                # selected trajectory cost (lower = better)
    behavior_state: str        # active FSM state

@dataclass
class Waypoint:
    x: float; y: float; yaw: float; velocity: float; timestamp: float
```

7.3 Event Bus

All inter-layer communication (except direct function calls within a tick) uses a lightweight publish/subscribe event bus. Each layer publishes its output to a named topic. The visualization layer subscribes to all topics. The replay logger subscribes to all topics.

Topic	Publisher
sensor/camera/front	Sensor Layer
sensor/lidar	Sensor Layer
perception/detections	Perception Layer
perception/lanes	Perception Layer
localization/ego_pose	Localization Layer
map/local_map	Environment Model
prediction/agents	Prediction Layer
planning/ego_trajectory	Planning Layer
control/vehicle_command	Control Layer

8 Technology Stack

Component	Technology	Version / Notes	Layer
Simulator	CARLA	0.9.15 — Unreal Engine 4.26	Simulation
Primary language	Python	3.10+	All layers
Deep learning	PyTorch	2.x + CUDA 12.x	Perception, Prediction
Object detection	Ultralytics YOLOv8	small/medium variants	Perception
LiDAR detection	PointPillars	OpenPCDet implementation	Perception
Lane detection	Ultra-Fast Lane Det. v2	PyTorch implementation	Perception
Semantic seg	SegFormer-B0	HuggingFace transformers	Perception
Tracking	SORT	NumPy + SciPy	Perception
Numerical compute	NumPy / SciPy	Latest stable	All layers
LiDAR visualization	Open3D	0.17+	Visualization
Primary rendering	Pygame	2.x — bird's-eye + HUD	Visualization
Camera overlay	OpenCV	4.x	Visualization
Data logging	HDF5 via h5py	Compressed replay archives	Replay
Event bus	ZeroMQ (pyzmq)	Or in-process Python queues	Inter-layer
Config files	YAML (PyYAML)	Sensor + stack config	Config
Scenario files	JSON	Scenario definitions	Scenario System
Web dashboard (opt.)	React + Three.js	Via WebSocket / FastAPI	Visualization
Web server (opt.)	FastAPI	WebSocket stream server	Visualization

Evaluation	pytest + custom metrics	Per-scenario eval harness	Evaluation

9 Implementation Milestones

Phase 1 — Simulator Setup and Vehicle Control (Weeks 1–2)

Goal: Ego vehicle drives through CARLA maps under manual scripted control. Full sensor suite attached and streaming.

- Install and configure CARLA 0.9.15. Verify UE4 rendering. Confirm Python carla client connects.
- Implement EgoVehicle class: spawns vehicle, attaches sensors per YAML config, exposes apply_control(throttle, steer, brake).
- Implement SimulationManager: tick loop, NPC TrafficManager config, weather setting, actor lifecycle.
- Implement ScenarioRunner: load JSON config, spawn actors, set routes, handle triggers.
- Implement waypoint follower (open-loop control): ego follows a fixed route for initial testing.
- Implement ReplayLogger: subscribe to all topics, write HDF5 archive per run.
- Deliverable: Ego drives SC-01 (Highway Cruise) under scripted waypoint control. All 7 sensors streaming. Run recorded and replayable.

Phase 2 — Sensor Integration (Week 3)

Goal: All sensor streams flowing into the stack with correct timestamps and formats.

- Implement SensorManager: camera, LiDAR, radar, GNSS, IMU callbacks → publish to event bus.
- Implement LiDAR preprocessing: motion compensation using IMU, ground plane removal (RANSAC), voxel downsampling.
- Implement camera preprocessing: undistort, resize to model input dimensions.
- Implement sensor synchronization: align camera and LiDAR frames to the same CARLA tick.
- Deliverable: All sensor data displayed raw in a basic Pygame window. Latency of sensor callbacks measured and logged.

Phase 3 — Perception Pipeline (Weeks 4–6)

Goal: All perception modules running in real time. Detections validated against CARLA ground truth.

- Implement and integrate YOLOv8 object detector (vehicles, cyclists, pedestrians, cones, traffic lights).
- Implement PointPillars LiDAR 3D detector. Fuse with 2D detections via projection.
- Implement SORT tracker with track lifecycle management.
- Implement Ultra-Fast Lane Detection v2.
- Implement SegFormer-B0 drivable space estimation.
- Implement cone detection with LiDAR depth association.
- Evaluate: Run all detectors against CARLA ground truth JSON. Compute precision, recall, mAP per class. Target mAP ≥ 0.75.
- Deliverable: Full perception pipeline running at ≤60ms per frame. Evaluation report with per-class metrics.

Phase 4 — Environment Model and Localization (Week 7)

Goal: LocalMap populated in real time from perception outputs and static map.

- Implement OpenDRIVE parser: extract lane graph from CARLA map export.
- Implement LocalizationLayer: GNSS → lane graph alignment, Frenet coordinate computation.

- Implement EnvironmentModel: merge agent detections + lanes + cones + signals into LocalMap.
- Implement cone accumulation and temporary lane boundary inference.
- Implement lane closure detection logic.
- Deliverable: LocalMap displayed in visualization with correct agent positions, lane overlays, and cone boundaries for SC-06 (Construction Zone).

Phase 5 — Prediction and Planning (Weeks 8–10)

Goal: Ego vehicle navigates all 7 scenarios under full AI control.

- Implement kinematic prediction model (constant velocity baseline).
- Implement behavior FSM with all 9 states.
- Implement Frenet polynomial trajectory generator with N=50 candidate sampling.
- Implement cost function with all 6 weighted terms.
- Implement Stanley lateral controller and PID longitudinal controller.
- Implement emergency brake override.
- Run all 7 scenarios. Measure completion rate, collision count. Tune FSM thresholds and cost weights.
- Deliverable: Completion rate ≥ 85% across all scenarios over 10 runs each. Evaluation report.

Phase 6 — Visualization Dashboard (Weeks 11–12)

Goal: Full visualization dashboard running at ≥20 FPS with all overlays active.

- Implement Pygame bird's-eye renderer: road, lane lines, agents, cones, predicted arcs, ego path.
- Implement camera panel: OpenCV overlay of bounding boxes, lane lines, traffic signals, drivable space.
- Implement LiDAR panel: Open3D or Pygame point cloud (height-mapped).
- Implement HUD overlay: speed, behavior state, latency, scenario name, sensor status.
- Implement keyboard interaction controls (all keys from Section 6.5).
- Implement replay playback: load HDF5 archive, scrub through frames, inspect any stack output.
- Deliverable: Dashboard running at ≥20 FPS on RTX 3080. All overlays active simultaneously. Replay working for all 7 scenarios.

Phase 7 — Scenario System and Evaluation Harness (Week 13)

Goal: Full scenario system operational. All 7 scenarios tested, evaluated, and documented.

- Finalize ScenarioRunner with full JSON schema support and all trigger types.
- Implement EvaluationHarness: load eval criteria from scenario JSON, compute metrics per run, output CSV report.
- Run automated evaluation: 20 runs per scenario, compute mean completion rate, collision rate, latency stats.
- Write scenario authoring guide: how to add a new scenario in < 1 hour.
- Deliverable: All 7 scenarios passing success criteria. Evaluation CSV reports. Authoring guide.

10 Future Extensions

10.1 Reinforcement Learning Driving Agent

Replace the hand-crafted FSM + polynomial planner with an RL agent trained entirely in CARLA. The modular architecture supports this: swap the Planning Layer implementation without touching any other layer.

- Observation space: LocalMap bird's-eye grid + EgoPose + LiDAR BEV image.
- Action space: Continuous (steer, throttle/brake) or discrete macro-actions (LANE_KEEP, MERGE_LEFT, STOP).
- Reward: +1 per meter progress, −100 collision, −10 traffic violation, −0.5 per unnecessary brake.
- Algorithm: PPO or SAC via Stable-Baselines3. Training on randomized versions of all 7 scenarios.

10.2 Large-Scale Traffic Simulation

Scale the NPC population from O(10) vehicles to O(1000) vehicles using CARLA's ScenarioRunner + SUMO co-simulation bridge. Enables study of emergent traffic behavior, bottleneck formation, and multi-agent planning.

10.3 Dataset Generation for Machine Learning

The platform already logs all sensor data and ground truth per frame. Extend ReplayLogger to export:

- COCO-format object detection dataset: camera images + 2D bounding box annotations from CARLA ground truth.
- nuScenes-format dataset: synchronized camera + LiDAR + 3D bounding boxes + ego trajectory.
- Lane dataset: camera images + lane polyline annotations.
- This positions the platform as a synthetic data generation tool for training real-world models.

10.4 Real-World Dataset Integration

Adapt the perception layer to run against real-world datasets (nuScenes, Waymo Open Dataset, KITTI) by replacing CARLA sensor callbacks with dataset loaders that emit the same typed SensorFrame events. The rest of the stack runs unchanged, enabling direct comparison of CARLA-trained models on real data.

10.5 Multi-Agent Autonomous Systems

Run multiple ego vehicles simultaneously in the same CARLA world. Each runs an independent autonomy stack. Vehicles communicate via a V2X simulation layer (shared LocalMap updates). Enables study of cooperative lane changes, intersection negotiation, and platooning.

10.6 Remote Operation Dashboard

Deploy the optional React + Three.js dashboard as a web-accessible interface. Add:

- Remote scenario launch and configuration via web UI.
- Live streaming of all visualization panels via WebRTC or MJPEG.
- Operator takeover: remote keyboard/gamepad control injected into the Control Layer.
- Multi-vehicle monitoring: switch between ego vehicles in a multi-agent run.

10.7 Advanced Perception Models

- Replace YOLOv8 + PointPillars with a unified multimodal transformer (e.g., BEVFusion) for joint camera-LiDAR perception.
- Add occupancy grid prediction (OccNet-style) for unstructured obstacle representation.
- Add online map prediction (MapTR-style) to reduce dependence on pre-built HD maps.

End of Document — CARLA Autonomy Demo Platform PRD v1.0